iDEFT: Intelligent Backup-Node Selection for Fault-Tolerant Network Function State Replication

Abstract—**Network Function Virtualization (NFV) systems that provide fault tolerance through state replication (OpenNF, Pico Replication, FTMB, REINFORCE, and DEFT among them) all replicate a primary network function's (NF) state to a backup instance. None of them specify how that backup instance is chosen. In DEFT, for example, the Failure Detection Unit simply assigns a new secondary NF with no stated criterion. This paper closes that gap with iDEFT (intelligent DEFT). We design three backup-selection strategies. Each computes the primary-to-backup mapping DEFT already consumes, so it replaces the static, hand-assigned mapping with no other change to the system. The first is a globally optimal assignment under per-backup capacity constraints, solved exactly via the Hungarian algorithm. Because it requires centralized computation, we use it only as an upper-bound benchmark, not a deployment candidate. The second is a multi-criteria greedy heuristic scoring health, remaining capacity, network latency, and hop-count. The third is a Proximal Policy Optimization (PPO) reinforcement-learning policy trained on the same objective with invalid-action masking. We evaluate all three against DEFT's static mapping, naive load-spreading baselines, and decentralized hash-based placement (rendezvous and consistent hashing). All methods drive DEFT's real, unmodified two-phase-commit (2PC) backup implementation under controlled concurrent load, over *two transports*: DEFT's shipped XML-RPC path, and gRPC over HTTP/2 with persistent channels, which is representative of production NFV replication. Holding every other variable fixed across the two lets us separate what is a property of backup placement from what is a property of the RPC stack. We test across four topology scales (9, 16, 32, and 128 hosts) under a run-level statistical design: one independent observation per (context seed $\times$ trial) execution, compared by paired Wilcoxon signed-rank tests with Holm-Bonferroni correction, matched-pairs rank-biserial effect sizes, and cluster-bootstrap confidence intervals that resample runs rather than individual calls. We find that feature-aware methods reliably avoid degraded backup hosts and substantially reduce service latency compared to blind load-balancing once hosts differ in quality, and that this holds under both transports: at 32 hosts the heuristic reduces mean latency by 53.7% against round-robin and 89.1% against a concentrating policy ($r = 0.994$, $p_{\mathrm{Holm}} < 0.0001$, 25 paired runs), and at 16 hosts by 31.8% and 77.1%. At homogeneous scale, where there is nothing to discriminate between candidates, both transports agree on a null to within 0.06 ms. A well-designed deterministic heuristic method tracks the Hungarian-optimal bound closely at every scale tested, remaining statistically indistinguishable from it throughout. We run a systematic sensitivity analysis over the heuristic's scoring weights and load-decay mechanics, under two models of what it means for a host to be degraded. Under an all-or-nothing model, in which degraded hosts sit at an extreme on every feature at once, all 287 weight configurations avoid them, which tells us about the benchmark rather than the method. Under a harder model with continuous severity and five distinct failure kinds, 80 and 111 of 286 configurations fail at 32 and 128 hosts, and one kind defeats the published scoring function outright: a host degraded only in its network path reads healthier than average on every resource gauge. Scoring against measured round-trip time rather than topology-derived hop latency restores the guarantee, and reveals that the latency term, not health, is what buys degraded-host avoidance. A learned RL policy (PPO) matches the feature-aware methods up to 32 hosts; at 128 hosts it avoids degraded hosts completely but load-balances less tightly than the heuristic. A cross-algorithm check with DQN shows the same concentration behavior at scale, and more sharply, under the same widened-exploration remedy, indicating the gap follows from the problem formulation rather than from one algorithm. An ablation of the reward's anti-concentration term shows it delivers a real but modest benefit that saturates almost immediately, and does not close the gap to the untrained heuristic. The two-transport comparison also yields a methodological result. Because DEFT's backup is an HTTP/1.0 `SimpleXMLRPCServer` with a five-deep accept backlog, every replication call opens a fresh TCP connection and the latency tail collapses onto Linux's SYN-retransmission timer rather than onto queueing delay; under gRPC the tail is ordinary queueing, carrying $240\times$ less mass beyond one second. The shipped transport inflates the measured placement effect by a consistent factor of about $4.3\times$, because it converts a degraded host's modest per-call service penalty into accept-queue overflow. Studies evaluating placement over connection-per-call RPC are subject to the same inflation (§V-A). We further report that a single machine cannot support a trustworthy latency measurement at 128 hosts on either transport, and confine that scale to structural results. The code, trained models, raw per-call records, and the saved output of every experiment are available for reproduction at https://bijoy111.github.io/ideft/.**

Index Terms—**network function virtualization, fault tolerance, state replication, backup node selection, replica placement, load balancing, reinforcement learning**

I. Introduction

Network Function Virtualization (NFV) replaces dedicated hardware middleboxes with virtualized network functions (NFs) running on commodity servers, enabling elastic scaling and fault tolerance that fixed hardware cannot provide [1]. A stateful NF's correctness depends on preserving per-flow and global state across failures and load-driven scaling events. The dominant approach to fault tolerance for stateful NFs is *replication*: a primary NF instance forwards its state updates to one or more backup (secondary) instances, which take over processing if the primary fails [2]–[4]. DEFT [5] extends this line of work with a distributed, controller-free two-phase-commit (2PC) protocol that achieves both elastic scaling and fault tolerance with strong consistency on global state updates, without the single point of failure a central controller introduces.

A. The Gap: Selection of Backup Nodes

Across this entire line of work, backup assignment is never considered as a decision with consequences. REINFORCE [4] designates a predecessor node and a backup node per NF without describing a selection rule. OpenNF [6], Split/Merge [7], StatelessNF [8], and S6 [9] all describe how state is migrated or shared once a target instance is known, never how that target is chosen in the first place. In DEFT's own reference implementation, this manifests concretely: the shipped static mapping for its 9-host evaluation topology concentrates 3 of 5 primaries onto a single backup host, with no load or health awareness whatsoever [5].

This is a consequential gap. The entire point of replication for fault tolerance is that the backup must be available and capable of taking over. If the backup itself is overloaded, unhealthy, or already serving many other primaries, the guarantees of the replication mechanism are only as good as an arbitrary assignment allows. Elastic scaling compounds this: as NF instances are added and removed under varying load, a backup pool's composition changes continuously, and a static or naively-assigned mapping cannot track it. We deliberately scope this work to the fault-tolerance half of state management, namely replication, because that is where the backup-selection decision lives. Choosing a target for state migration during elastic scaling is a related but distinct decision; our assignment formulation could extend to it, but we do not evaluate it here.

B. Contributions

We present iDEFT (intelligent DEFT), addressing backup-node selection directly and using DEFT as the concrete system under study though the problem and our methods generalize to any primary/backup replication scheme. Specifically, we:

1) Formulate backup selection as a feasibility and load-aware assignment problem, and design three methods: an exact globally-optimal assignment via the Hungarian algorithm under capacity constraints, a deterministic multi-criteria greedy heuristic, and a learned PPO policy with invalid-action masking during training.
2) Build a reproducible evaluation harness that drives DEFT's real, unmodified 2PC backup implementation under controlled concurrent load via XML-RPC (§IV), and evaluate every method against DEFT's shipped static mapping, naive baselines (random, round-robin and a fairer randomized-offset variant), and decentralized hash-based placement (rendezvous hashing and weighted consistent hashing, the NFV analog of CRUSH/Dynamo), across four topology scales, multi-hub topologies, and four metrics (service latency, throughput, failure rate, selection overhead) under a run-level paired statistical design (§IV-D).
3) Show that feature-aware selection reliably avoids degraded hosts and reduces mean and p95 service latency once hosts differ in quality, that the simple deterministic heuristic tracks the Hungarian-optimal bound at every scale, and, through a systematic sensitivity analysis over the heuristic's scoring weights and load-decay constants (§V-G), that these constants are principled, near-optimal choices rather than arbitrary ones.
4) Treat the transport as a controlled variable rather than a fixed property of the testbed (§V-A), evaluating every method over both DEFT's shipped XML-RPC path and gRPC over HTTP/2. This separates placement effects from RPC-stack effects, shows the central finding holds under both, and quantifies how much the shipped transport inflates it. We additionally characterize two limits of the harness: absolute latencies drift between identical executions as ephemeral-port TIME_WAIT pressure accumulates, and a single machine cannot resolve latency at 128 hosts on either transport.
5) Report the measured behavior of RL on this problem: PPO matches the feature-aware methods up to 32 hosts and, at 128 hosts, avoids degraded hosts completely while load-balancing less tightly than the heuristic. A cross-algorithm check with DQN (§V-F) reproduces the same concentration behavior at scale under the same remedy, which points at the problem formulation rather than at a single algorithm.

II. Related Work

A. Elastic and Fault-Tolerant State Management

NF state management for elastic scaling follows two strategies. State migration moves NF state between instances. Split/Merge [7] splits state into per-flow and global parts; global updates merge periodically and stay eventually consistent, but in-flight traffic can be dropped during migration. OpenNF [6] offers move, copy, and share operations for different consistency levels. Both systems need a central controller to coordinate migration.

Migration avoidance instead decouples state from processing. StatelessNF [8], [10] keeps state in a remote data store, adding per-packet latency and a single point of failure. S6 [9] spreads state across a Chord-like key/object layer [11], reducing, but not eliminating, migration through caching and replication.

NF fault tolerance instead relies on replication and checkpointing. Pico Replication [2] checkpoints frequently [12] for loss-free recovery, but does not handle in-flight packets arriving during a checkpoint. FTMB [3] checkpoints infrequently and logs and replays packets instead, favoring throughput over checkpoint frequency. REINFORCE [4] combines both: packets are buffered at a predecessor node and replayed to a backup on failure, with lazy checkpointing except for non-deterministic packets [13], which force a strict checkpoint. All three assume a backup already exists for each primary; none describe how it is chosen.

DEFT [5] spans both strands: one distributed 2PC protocol handles elastic scaling and fault tolerance together, with no central controller (unlike [6], [7]). But it inherits the fault-tolerant systems' gap: backup assignment is a bookkeeping step in its Failure Detection Unit, not a studied decision.

TABLE I

HOW PRIOR NF STATE-MANAGEMENT SYSTEMS CHOOSE THE TARGET HOST. "MECHANISM" IS WHAT THE SYSTEM SPECIFIES FOR MOVING OR REPLICATING STATE; "TARGET SELECTION" IS HOW THE DESTINATION IS CHOSEN. EVERY SYSTEM SPECIFIES THE FORMER IN DETAIL; NONE SPECIFIES THE LATTER AS A CRITERIA-DRIVEN DECISION. THE STORAGE SYSTEMS IN THE LAST TWO ROWS DO, WHICH IS WHY WE ADOPT THEIR MECHANISM AS A BASELINE FAMILY (§V-H).

System	Purpose	Mechanism	Target selection
Split/Merge [7]	Elastic scaling	Split per-flow/global state	Controller-assigned; unspecified
OpenNF [6]	Elastic scaling	Move/copy/share	Controller-assigned; unspecified
StatelessNF [8]	Scaling + FT	External state store	N/A (no per-NF target)
S6 [9]	Elastic scaling	Chord-like DHT	Hash of key, quality-unaware
Pico Rep. [2]	Fault tolerance	Frequent checkpointing	Backup assumed to exist
FTMB [3]	Fault tolerance	Checkpoint + replay	Backup assumed to exist
REINFORCE [4]	Fault tolerance	Buffer at predecessor	Designated; no stated rule
DEFT [5]	Scaling + FT	Distributed 2PC	Static map in Failure Detection Unit
Cand-RL [14]	SFC placement	PPO	Criteria-driven, but for *primaries*
DDQN-VNFPA [15]	VNF placement	Double DQN	Criteria-driven, but for *primaries*
CRUSH [16]	Storage replicas	Pseudo-random hash	Failure-domain and weight aware
Dynamo [17]	Storage replicas	Consistent hashing	Capacity-weighted preference list
iDEFT	**Fault tolerance**	**DEFT's 2PC, unmodified**	**Health, capacity, measured RTT, hops**

What the table shows, and the insight we draw from it. Table I separates two decisions that the NFV state-management literature consistently conflates: *how* state reaches another host, and *which* host it reaches. Every system in the first two groups specifies the mechanism in considerable detail, and none specifies the selection as anything more than an assumption, a controller's prerogative, or a static map. The RL-for-NFV work does make placement criteria-driven, but places *primaries*: it decides where a function should run, not which already-running host should back it up. Distributed storage settled the analogous question long ago, and the last two rows show what a criteria-driven answer looks like.

The insight behind our design follows from that gap rather than from any single technique. Backup selection is not primary placement with a different label: the candidate set is the set of hosts *already running*, so the decision is constrained by the live resource state of machines that are simultaneously serving their own traffic, and it must be cheap enough to redo whenever the pool changes. That rules out the centralized optimizers used for primary placement as deployment candidates, which is why we use the Hungarian assignment only as a bound (§III). It also explains why a single coordinated greedy pass beats the decentralized weighted-hash schemes that storage systems rely on (§V-H): hashing spreads load only statistically, whereas one pass with an inter-primary load penalty spreads it by construction, and a backup pool of tens of hosts is small enough that the coordination is affordable. The combination is deliberately conventional in its parts; what is new is treating the decision as one worth making at all, and establishing empirically what it is worth.

B. Reinforcement Learning for NFV Resource Management

A separate line of work applies reinforcement learning to NFV resource management, almost entirely for primary placement, scaling, or embedding, not backup/replica selection. Cand-RL [14] uses PPO for availability-aware Service Function Chain (SFC) placement. DDQN-VNFPA [15] uses Double Deep Q-Networks for VNF placement. Two recent surveys, on RL for virtual network embedding [18] and on RL for network slice scaling and placement [19], both catalogue placement, scaling, and embedding as RL's NFV subproblems; backup/replica selection for fault tolerance appears in neither.

C. Replica or Backup Selection

Where the elastic and fault-tolerant state-management systems above specify how state is replicated or migrated, they take the choice of backup host as given, left to an implicit default rather than chosen from the candidate hosts' resource state. The few efforts that come near replica-like selection approach it from a different angle. Kibalya et al. [20] place stateful VNFs, but the state guiding that placement is a primary's own execution state, not a fault-tolerance relationship between a primary and a distinct backup. Avgeris et al. [21], the closest in spirit, reposition VNFs once a failure or contention event has already been detected, a reactive remedy applied after the fact, whereas we choose a secondary proactively, as replication is being established.

The idea that replica placement deserves algorithmic attention in its own right is, by contrast, long established outside NFV, in distributed storage. CRUSH [16] computes failure-domain and load-aware placement without a central lookup table, and Dynamo [17] spreads replicas via consistent hashing with capacity-aware preference lists; both make "which node holds the copy" a first-class, criteria-driven decision. Their underlying mechanism, decentralized weighted hashing (rendezvous/HRW and consistent hashing), carries over directly to NF backups, which is why we adopt it as a baseline family (§V-H). We find that such stateless schemes trail a lightly-coordinated selector on load-balancing. To our knowledge, no prior work brings this first-class treatment to *proactive backup-NF selection* for stateful NFV fault tolerance, the gap this paper addresses.

III. BACKUP-SELECTION METHODS

Let P be the set of primary NFs and H the set of all hosts (a superset of P; any host not currently a primary for a given flow, is a feasible backup for that primary, including hosts that are primaries for other flows). For each primary $p \in P$ we must choose a backup $b \in H \setminus \{p\}$. Each candidate host h has a dynamic resource state: health $\mathrm{health}(h) \in [0, 1]$, remaining capacity $\mathrm{cap}(h)$ out of $\mathrm{cap}_{\max}(h)$, CPU and storage availability, and a topology-derived hop-count and latency to p. Concretely, per host and per random seed these are drawn independently as $\mathrm{cap}_{\max} \sim \mathcal{U}\{50, \ldots, 100\}$ (integer), with cap initialised to $\mathrm{cap}_{\max}$; $\mathrm{health} \sim \mathcal{U}[0.5, 1.0]$; and CPU and storage availability each $\sim \mathcal{U}[0.4, 1.0]$; latency is $1.5\,\mathrm{ms} \times \mathrm{hops}$. Degraded hosts in the heterogeneous scenarios (§IV) are constructed under one of two models (§V-G). The *binary* model, used for the latency experiments and for all previously published results, pins $\mathrm{health} = 0.05$, $\mathrm{cap} = 2$, $\mathrm{cap}_{\max} = 60$ (capacity ratio ≈ 0.033). The *realistic* model draws a continuous severity and one of five failure kinds per host, leaves the untouched features at their healthy draw, and gives healthy hosts a baseline utilisation so that remaining

capacity is a continuous measurement rather than a flag. Infrastructure hosts (client, stamper) are never sampled as backups.

From synthetic features to real telemetry. These four features are deliberately chosen to correspond to signals a production NFV orchestrator already exposes, so the abstraction is a stand-in for measurable metrics rather than an idealization unavailable in practice. health maps to a composite liveness/health score: failure-detector confidence, recent error/drop rate, or hardware SMART/thermal indicators, as surfaced by an ETSI NFV-MANO VNF health indication or a Kubernetes readiness/liveness probe. The capacity ratio $\mathrm{cap}/\mathrm{cap}_{\max}$ maps to headroom in the NF's dominant scaling resource: free flow-table or connection-tracking entries, session/NAT-binding slots, or admission-control headroom against a provisioned limit. CPU and storage availability map to standard hypervisor/OS utilization ($1 - \mathrm{util}$, free state-store fraction) exported by cAdvisor, node-exporter, or libvirt; hop-count and latency map to the SDN controller's topology view and active RTT probing. Validating the selection against such real workload telemetry, rather than the synthetic draws used here, remains future work (§VI).

A. Baselines

Static (DEFT's shipped mapping). The hand-assigned mapping DEFT ships with for its 9-host reference topology, regenerable on demand for direct comparison.

Concentrated. Every primary assigned to the same single feasible backup: a scale-independent, maximally naive worst case, generalizing DEFT's own static mapping's tendency to over-concentrate load onto one host.

Random. A uniformly random feasible backup per primary, seeded for reproducibility: a sanity floor any non-random method should beat.

Round-robin. Cycles through the sorted list of all feasible hosts, skipping a candidate only when it equals the primary itself. Load-balances blindly, with zero awareness of host health, capacity, or latency. We also evaluate a *randomized-offset* variant that begins the cycle at a random position per run, so its degraded-host exposure reflects the degraded fraction rather than an artifact of the fixed host ordering (§V-H).

Rendezvous hashing (HRW) and consistent hashing. *Plain HRW* assigns each primary to the feasible backup with the highest deterministic hash weight $\mathrm{hash}(p, c)$: fully decentralized (no coordinator, no cross-primary state) and load-spreading in expectation, but health/capacity-unaware, the NFV analog of a consistent-hashing replica placement. *Weighted HRW* weights the hash by $\mathrm{health}(c) \cdot \mathrm{cap}(c)/\mathrm{cap}_{\max}(c)$ (in the spirit of CRUSH/Dynamo node weights), making it quality-aware while remaining decentralized. *Weighted consistent hashing* is a second decentralized quality-weighted point: a Dynamo-style hash ring with capacity/health-proportional virtual nodes (a host whose weight rounds to zero, e.g. a degraded host, places none and is never selected). Unlike the heuristic, none of these apply an inter-primary load penalty, so they spread only statistically.

B. Optimal: Capacity-Constrained Hungarian Assignment

To obtain an upper bound the heuristic and RL policy can be judged against, we solve the assignment problem exactly, using the same multi-criteria score $\mathrm{score}(p, c)$ defined in the next subsection (Eq. (1): a weighted combination of health, remaining capacity, latency, and hop-count). Each feasible backup h is expanded into $k = \lceil |P|/|H_{\mathrm{feas}}| \rceil$ capacity-constrained columns, where H_{feas} is the set of backup-eligible hosts (all hosts except the client and stamper); the cost-matrix entry for primary p and column j (backup h) is $-\mathrm{score}(p, h)$, or a large fixed penalty (10^6) if h is infeasible for p (including $h = p$). Because backup-eligible hosts always outnumber primaries in our topologies, $k = 1$ at every scale evaluated here ($\lceil 5/7 \rceil$, $\lceil 10/16 \rceil$, $\lceil 20/32 \rceil$, $\lceil 80/128 \rceil$ for the 9/16/32/128-host topologies, all equal to 1), so the solver assigns each primary to a distinct backup and the optimum attains a uniform maximum load of one; the $k > 1$ machinery is retained for denser regimes in which primaries would outnumber feasible backups. We solve this exactly via the Hungarian algorithm [22] (`scipy.optimize.linear_sum_assignment` [23]), yielding a globally optimal assignment under the capacity constraint, something no greedy or online method can guarantee in general. Solving it requires the complete cost matrix over all primaries and candidate backups at once, which in turn requires a party with a global view to gather that state and run the solver before any assignment is made. This centralized computation is in tension with DEFT's distributed, controller-free design (§I), so we use the Hungarian assignment strictly as an upper-bound benchmark, not as a deployment candidate.

C. Heuristic: Multi-Criteria Greedy Scoring

For each primary (processed in a fixed order), the heuristic scores every feasible candidate c as

$$\mathrm{score}(p, c) = w_h \cdot \mathrm{health}(c) + w_k \cdot \frac{\mathrm{cap}(c)}{\mathrm{cap}_{\max}(c)} - w_\ell \cdot \hat{\ell}(p, c) - w_d \cdot \hat{d}(p, c) \tag{1}$$

where $\hat{\ell}$ and $\hat{d}$ are the min-max-normalized latency and hop-count from p to c among p's candidates, and default weights $(w_h, w_k, w_\ell, w_d) = (0.35, 0.35, 0.20, 0.10)$ favor health and capacity, which are what actually differentiate candidates once a topology has heterogeneous backup quality. The greedy assignment picks $\arg\max_c \mathrm{score}(p, c)$ for each primary in turn, then applies a load penalty to the chosen host ($\mathrm{cap} \mathrel{-}= 20$; $\mathrm{health} \mathrel{*}= 0.95$) before moving to the next primary. This penalty is the mechanism that spreads load across the backup pool without any learning component.

D. RL: PPO with Invalid-Action Masking

We train a Proximal Policy Optimization [24] agent (`stable-baselines3` [25] over a `gymnasium` [26] environment) on the same scoring objective as the heuristic. One episode assigns a backup to every primary in sequence, under

TABLE II
PPO TRAINING CONFIGURATION PER MODEL. UNLISTED SETTINGS ARE STABLE-BASELINES3 DEFAULTS. $[a, a]$ DENOTES A TWO-HIDDEN-LAYER MLP POLICY AND VALUE NETWORK OF WIDTH a.

Model (scale)	Timesteps	ent. coef	net arch	n_{steps}	batch
9-host	50 k	0.01	$[128, 128]$	2048	64
16-host	80 k	0.01	$[128, 128]$	2048	64
32-host	100 k	0.01	$[128, 128]$	2048	64
128-host (collapsed)	400 k	0.00	$[64, 64]$	2048	64
128-host (intermediate)	600 k	0.01	$[128, 128]$	2048	64
128-host (final, reported)	1 M	0.02	$[256, 256]$	4096	256

one random resource realization; the observation is each host's $[\text{health}, \text{cap}/\text{cap}_{\max}, \text{cpu}, \text{storage}]$ concatenated with a one-hot encoding of the currently-active primary, and the action space is $\text{Discrete}(|H|)$ (observation dimension $4|H| + |H|$). The reward is $\text{score}(p, c)$ for a feasible choice, minus an overload penalty of -2 for each primary already assigned to that backup (i.e. $-2(\ell - 1)$ when the choice makes host c the ℓ-th primary's backup), and a large fixed penalty (-10) for an infeasible choice, so invalid actions are discouraged *during training*, not merely corrected at inference time as in a naive post-hoc-masking design. A fresh random resource realization is drawn every episode so the learned policy generalizes across resource realizations rather than memorizing one instance. At inference, the deterministic policy is rolled out once per primary, falling back to the best-scoring feasible candidate in the rare case the policy still selects an infeasible action.

Table II lists the full per-model training configuration; all other PPO settings are `stable-baselines3` defaults (learning rate 3×10^{-4}, $\gamma = 0.99$, GAE $\lambda = 0.95$, clip range 0.2, 10 optimization epochs per update). Each model is trained for its own host count, so a differently-sized topology requires a freshly trained model (§VI); the two 128-host variants are the collapsed default-hyperparameter baseline and the reported fix of §V-E, both released as checkpoints. The 128-host *intermediate* model (which only partially improved the spread) is listed for completeness but not shipped. §V-F additionally trains a DQN agent on this same environment and reward, as a cross-algorithm check on the load-balancing behavior PPO exhibits at large host counts.

IV. EXPERIMENTAL METHODOLOGY

A. Testbed: RPC-Isolation, Not Physical Mininet

We measure backup-selection quality by driving the *real, unmodified* `Backup` and `Primary` 2PC classes from DEFT's implementation directly over XML-RPC, with each backup host running as its own OS process on localhost. This is a deliberate departure from a physical Mininet network path, which we attempted first: the development environment's kernel lacked the `openvswitch` kernel module and the `htb` traffic-control queueing discipline, no SDN controller was included in the original repository, and, decisively, `TCLink` bandwidth throttling did not engage on the available kernel, so there was no real resource constraint for contention to manifest against over the physical network path. The RPC-isolation testbed, by directly exercising the actual backup RPC server under controlled concurrent load, gives a more direct and reproducible measurement of the contention mechanism in question, at the cost of not validating full end-to-end network behavior (switching, real link bandwidth, packet loss); see §VI.

Each primary fires a *burst* of simultaneously in-flight RPC calls rather than one sequential call-then-wait stream, matching how concurrent primaries would load a shared backup in production. Three implementation pitfalls were identified and corrected during harness development: (1) running backup servers as threads within one process shares a GIL, confounding the very contention effect being measured, so servers now run as separate OS processes; (2) each burst worker holds one `ServerProxy` for all of its calls, with retry-with-jittered-backoff on transient connection resets. We initially described this as reusing one TCP connection per worker; it does not. DEFT's `Backup` inherits `SimpleXMLRPCServer`'s default `protocol_version` of HTTP/1.0, so the server closes the connection after every response and the proxy silently reconnects: we measure twenty calls through a single proxy producing twenty `accept()`s server-side. Every replication call therefore performs a fresh TCP handshake against a five-deep accept backlog, which is the direct cause of the tail behavior analysed in §V-A; and (3) an early version built a single host-resource context per seed and passed that same object to every method, but the heuristic and RL selectors apply an in-place load-decay penalty to each host they pick, so any method evaluated after them scored against an already-depleted context and its measured mapping depended on execution order, so each method now receives its own copy of the context.

A fourth issue surfaced only at the largest scale. `xmlrpc.client` applies no socket timeout, so a connection the server has accepted but is not reading stalls the client indefinitely. Because the backup is single-threaded, one such connection blocks its accept loop and every other client queues behind it: at 128 hosts with 80 primaries on one backup we observed a run frozen for 80 s with the server's CPU time, its established-connection count, and its SYN-SENT backlog all constant, making no forward progress at all. We therefore bound each call with a 20 s client-side socket timeout, well above the largest genuine service time observed anywhere in these experiments (7.2 s), so it breaks the deadlock without censoring real slow calls. As with the retry budget, this changes the benchmark client, not the backup server under test.

B. The Second Transport

Alongside DEFT's shipped XML-RPC path we implement the same replication call over gRPC: a service exposing exactly the method DEFT's backup exposes (`update_state(state)` returning an acknowledgement), one persistent HTTP/2 channel per primary rather than a connection per call, and a server thread pool in place of a single accept loop. Message payloads carry the same flow-keyed state. The selection methods, scoring function, topologies, degraded-host injection, workload, retry policy and statistical

analysis are identical across the two arms, so the transport is the only thing that varies.

Two details keep the comparison honest. First, the gRPC server's worker-pool size is an explicit parameter rather than an inherited default, so the transport change can be separated from the concurrency change; setting it to one reproduces the XML-RPC backup's strict serialization over HTTP/2. Second, gRPC reports channel readiness exactly, so backup startup is synchronised rather than waited out with a fixed sleep, removing a source of spurious early-call errors present in the XML-RPC arm.

C. Scenarios

Homogeneous (9 hosts). DEFT's real reference topology, 5 primaries, synthetic resource values drawn independently per random seed with no engineered heterogeneity.

Scaled heterogeneous (16, 32, 128 hosts). Synthetic star topologies at increasing scale (10/20/80 primaries; 6/12/48 backup-eligible hosts), with $\lfloor |H \setminus P|/3 \rfloor$ of the backup-eligible hosts deliberately degraded (health $= 0.05$, capacity ratio ≈ 0.03, plus an artificial +50 ms per-call server-side delay simulating a genuinely overloaded machine). This isolates whether feature-aware methods avoid known-bad hosts, independent of load-balancing alone, and whether the finding holds as the candidate pool grows by roughly an order of magnitude.

D. Statistical Methodology

We draw multiple independent topology/resource realizations (context seeds) rather than repeating trials of one fixed instance, since repeated trials of a single instance only capture RPC-timing noise, not whether a result generalizes across problem instances.

The unit of analysis is a run, not a call. One *run* is one (method, context seed, trial) execution. The individual RPC calls within a run are not independent observations: a burst's calls share a TCP connection sequence, a single-threaded backup server process, an OS scheduler, and one resource realization. Treating each call as a sample inflates n into the tens of thousands and drives p-values to zero without adding evidence. The effect is easy to quantify: on synthetic data built from two methods with *no* true difference and correlated calls within each run, a Mann-Whitney U test pooled over 15,000 calls returns $p = 1.1 \times 10^{-44}$, while the run-level test on the same data correctly returns $p = 0.76$. We therefore reduce each run to per-run statistics (mean, median, p95 service latency, and failure rate over that run's own calls) and analyse those, giving $n = 15$ at every scale reported here.

Comparisons are paired. Every method is executed under each (context seed, trial) condition, so methods are compared by a two-sided Wilcoxon signed-rank test on the paired per-run differences, with the matched-pairs rank-biserial correlation $r = (T^+ - T^-)/(T^+ + T^-)$ as effect size; $r = \pm 1$ means every run agrees on the direction. Pairing matters here because the between-run variance is large and common-mode (§V-A), and pairing cancels it exactly. We apply Holm-Bonferroni correction across the full family of pairwise comparisons at each scale, and report cluster-bootstrap 95% confidence intervals (5000 resamples) that resample whole runs rather than individual calls. Per-trial method execution order is randomized so no method is systematically favored by background system load.

Failed calls are censored. A call that exhausts its retry budget contributes to the reported failure rate and to nothing else. Imputing a sentinel latency for it, as our earlier harness did, contaminates every percentile computed from the sample.

Reporting mean and p95, not p99. §V-A shows that p99 in this testbed measures the kernel's TCP SYN-retransmission timer rather than replication latency. We therefore report mean and p95 throughout, and use p99 only where the point is the tail pathology itself.

V. Results

A. Transport as a Controlled Variable

We evaluate every method over two transports. The first is DEFT's shipped path: its `Backup` is a `SimpleXMLRPCServer`, which is single-threaded, speaks HTTP/1.0, and listens with a backlog of five. The second is gRPC over HTTP/2, with one persistent channel per primary and a server thread pool, which is representative of how production NFV systems carry state replication. The two expose the same method, perform the same work per call, and are driven by the same selection methods, topologies, workloads and statistics; only the delivery mechanism differs. The transport is therefore a controlled variable, and a finding that holds under both is a property of backup placement rather than of one RPC stack.

This matters because the shipped transport distorts the measurement in a specific and quantifiable way.

On XML-RPC the latency tail is a kernel timer, not queueing delay. HTTP/1.0 means the server closes the connection after each response, so every replication call performs a fresh TCP handshake. We verified this directly: twenty calls issued through a single client proxy produce twenty server-side `accept()`s. Under burst load the five-deep backlog overflows, the SYN is dropped, and the client waits out the kernel's retransmission timer. The resulting distribution is sharply bimodal with an empty gap between the modes:

Hosts	<100 ms	100–500	**500–900**	900–1100	>1100
9	96.96%	0.355%	**0.000%**	2.164%	0.523%
16	82.33%	14.428%	**0.042%**	1.549%	1.650%
32	67.17%	29.129%	**0.095%**	1.011%	2.597%
128	47.71%	46.782%	**0.428%**	0.924%	4.157%

Queueing produces a smooth tail; this does not. The 900–1100 ms calls sit at 1052–1059 ms with a standard deviation of 24–29 ms at every scale, which is Linux's initial SYN-retransmission timeout, with the next retry contributing the ≈3 s cluster. Consequently p99 on this transport is a threshold statistic rather than a latency measurement: since 1–2% of calls land on the timer, it reads either tens of milliseconds or ≈1050 ms depending only on whether a run's SYN-drop rate crosses one percent. This also explains an artifact in an

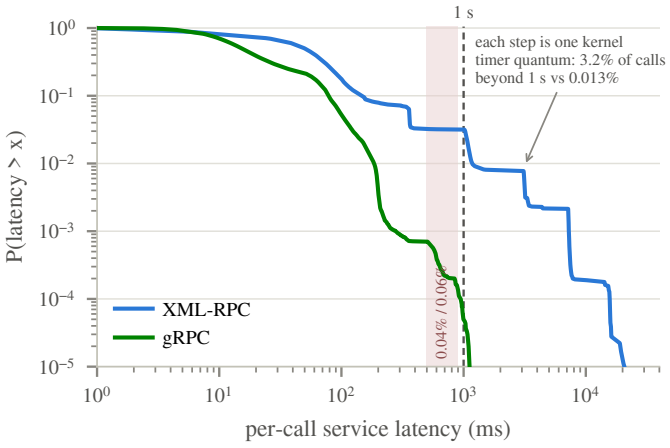


Fig. 1. Complementary CDF of per-call service latency at 16 hosts, by transport (log–log). The XML-RPC tail is a staircase of kernel timer quanta rather than a queueing curve; the gRPC tail decays smoothly and ends two orders of magnitude earlier. The shaded band is 500–900 ms.

earlier version of our 32-host experiment, where p99 appeared to improve by an order of magnitude; that run used a reduced protocol whose lower concurrency pushed the SYN-drop rate just below one percent.

Under gRPC the pile-up disappears. Figure 1 plots the complementary CDF of per-call service time for both arms at 16 hosts (180,000 and 300,000 calls of the same workload). The XML-RPC curve is a staircase: it runs flat and then steps down at each timer quantum, with **3.20%** of calls landing beyond 900 ms and a maximum of 22,773 ms. The gRPC curve decays smoothly, with **0.013%** beyond 900 ms, a 240× difference in tail mass, and a maximum of 1139 ms. Percentiles follow: p95 101.9 vs 360.6 ms, p99 184.9 vs 1175.8 ms, p99.9 277.9 vs 7261.6 ms.

The 500–900 ms band itself is sparsely occupied on *both* arms (0.042% and 0.057%), which is the point: on the gRPC arm that sparsity is simply the thin end of a decaying distribution, whereas on the XML-RPC arm it is a gap with 3.2% of the mass piled immediately beyond it. A queueing tail does not have that shape. p99 is a usable metric on the gRPC arm, and we report it there.

The transport amplifies the placement effect by a consistent factor. Table III compares the paper's central contrast, `round_robin` against `heuristic`, under both. The effect is real and significant on both transports at 16 and 32 hosts, and absent on both at homogeneous scale, where the two arms agree to within 0.06 ms (−2.08 vs −2.14). But its magnitude is roughly 4.3× larger on XML-RPC (48.05 vs 11.43 ms at 16 hosts; 43.69 vs 9.75 ms at 32). The mechanism is the backlog: a degraded host's added per-call service time pushes its accept queue into overflow, converting a modest service penalty into whole seconds of retransmission wait. In proportional terms the advantage is preserved or larger on gRPC (31.8% at 16 hosts under both; 53.7% versus 25.7% at 32), because baseline latencies fall by a similar factor.

This is worth stating as a methodological result in its own right: evaluating placement policies over a connection-per-call RPC stack inflates measured placement effects, because the transport converts a per-call service penalty into queue collapse. Studies built on comparable harnesses are subject to the same inflation.

TABLE III

ROUND_ROBIN VS. HEURISTIC UNDER BOTH TRANSPORTS. Δ IS THE MEDIAN PER-RUN DIFFERENCE (POSITIVE FAVOURS THE HEURISTIC). THE EFFECT IS REAL ON BOTH; IT IS ROUGHLY 4.3× LARGER ON XML-RPC, WHOSE ACCEPT-BACKLOG PATHOLOGY AMPLIFIES THE DEGRADED-HOST PENALTY.

	XML-RPC (n=15)			gRPC (n=25)		
Hosts	Δ (ms)	r	p_{Holm}	Δ (ms)	r	p_{Holm}
9 (homog.)	-2.08	-0.517	0.4163	-2.14	-0.422	0.5010
16	+48.05	+0.983	0.0012	+11.43	+0.705	0.0104
32	+43.69	+1.000	0.0009	+9.75	+0.994	<0.0001

Absolute latencies are not reproducible across executions; paired differences are. Because every XML-RPC call consumes an ephemeral port, TIME_WAIT entries accumulate rapidly: mid-session we measured 14,038 sockets in TIME_WAIT against an ephemeral range of 28,231 ports. Two independent executions of the identical 9-host configuration differed by roughly 17 ms in the mean of *every* method, while the paired differences between methods moved by about 1 ms. The drift is common-mode, so the paired design of §IV-D cancels it, but it sets a resolution floor: on this harness, differences below roughly 10 ms do not survive replication. We make no claim below that threshold. Four candidate effects of that size appeared significant in a single execution and vanished under replication, which is why every scale is run twice and the gRPC arm at $n = 25$.

B. Homogeneous Scale

Table IV reports the 9-host scenario, where resource values are drawn without engineered heterogeneity. The result is a clean null, and it replicates across transports: `round_robin`, `optimal`, `PPO` and `random` are all statistically indistinguishable from the heuristic, while `concentrated` and `baseline` are decisively worse ($r = -1.000$ and $r = -0.680$).

The reading is that where hosts are uniform, feature-awareness buys nothing, because there is nothing to discriminate between candidates. What still matters is not piling every primary onto one backup. This is the correct negative control for the experiments that follow: it shows the apparatus does not manufacture an advantage for feature-aware methods where none should exist.

C. Scaled Heterogeneous: 16 and 32 Hosts

Tables V and VI isolate feature awareness: one third of the backup-eligible hosts are deliberately degraded. `concentrated`, `heuristic`, `optimal` and `PPO` *never* route a primary to a degraded host in any seed at either scale; `round_robin` hits one on 2 of 10 and 4 of 20 assignments in every seed, cycling blindly with no way to know which

TABLE IV
HOMOGENEOUS SCALE (9 HOSTS, 5 PRIMARIES). XML-RPC: 15 RUNS; GRPC: 25 RUNS (5 CONTEXT SEEDS × 5 TRIALS), 200 CALLS/PRIMARY, BURST 10. EFFECT SIZES ARE PAIRED AGAINST `HEURISTIC` ON THE GRPC ARM.

	XML-RPC	gRPC			
Method	Mean (ms)	Mean (ms)	95% CI	r	p_{Holm}
`baseline`	73.45	20.58	[17.55, 24.56]	+0.680	0.0243
`concentrated`	107.42	32.60	[29.84, 35.77]	+1.000	<0.0001
`random`	58.70	16.04	[14.50, 17.75]	+0.329	0.7817
`round_robin`	47.51	14.06	[12.22, 16.43]	-0.422	0.5010
`heuristic`	51.10	14.72	[13.44, 16.18]	–	–
`optimal`	45.90	13.60	[12.50, 14.92]	-0.286	0.8799
`PPO`	48.44	13.79	[12.06, 16.21]	-0.403	0.5010

TABLE V
SCALED HETEROGENEOUS, 16 HOSTS (10 PRIMARIES, 2 DEGRADED).

	XML-RPC	gRPC			
Method	Mean (ms)	Mean (ms)	95% CI	r	p_{Holm}
`concentrated`	175.36	77.97	[72.12, 83.99]	+1.000	<0.0001
`random`	145.00	31.45	[24.34, 39.47]	+0.655	0.0185
`round_robin`	144.21	26.23	[24.08, 29.12]	+0.705	0.0104
`heuristic`	98.35	17.88	[14.03, 22.76]	–	–
`optimal`	90.73	18.82	[14.30, 24.45]	-0.046	1.0000
`PPO`	89.38	13.83	[12.57, 15.49]	-0.200	1.0000

hosts are bad; `random` is inconsistent, hitting between none and four depending on the seed.

The latency consequence is unambiguous and holds on both transports. On the gRPC arm at 32 hosts the heuristic beats `round_robin` by 9.75 ms at the median run with $r = 0.994$ and $p_{\text{Holm}} < 0.0001$, and beats `random` by 13.10 ms ($r = 0.945$); at 16 hosts the same comparisons give $r = 0.705$ ($p_{\text{Holm}} = 0.0104$) and $r = 0.655$ ($p = 0.0185$). In mean terms the heuristic reduces service latency by 53.7% against `round_robin` and 89.1% against `concentrated` at 32 hosts, and by 31.8% and 77.1% at 16. Against this, `optimal` and `PPO` are indistinguishable from the heuristic at both scales ($|r| \leq 0.20$, all $p_{\text{Holm}} = 1.000$): the cheap deterministic method attains the exact optimum's placement quality.

`random` deserves separate comment. Its mean is close to `round_robin`'s, but its confidence interval is far wider, and on the XML-RPC arm it is *not* significantly better than `concentrated`. Random placement is not merely mediocre on average; it is unstable, because whether a given draw lands on degraded hosts is uncontrolled. Figure 2 plots both arms across the full scale sweep; the p95 row shows the degraded-host penalty that `round_robin` and `random` pay at 16 and 32 hosts while the feature-aware methods stay flat.

D. Mechanism Check: Latency vs. Backup Load

Table VII pools service latency by `backup_load` (the number of primaries sharing a backup), aggregated to one observation per (run × backup) so that correlated calls against a single loaded server count once. Mean latency rises monotonically with load, isolating sharing itself, independent of which method produced it, as the mechanism driving latency under contention.

TABLE VI
SCALED HETEROGENEOUS, 32 HOSTS (20 PRIMARIES, 4 DEGRADED).

	XML-RPC	gRPC			
Method	Mean (ms)	Mean (ms)	95% CI	r	p_{Holm}
`concentrated`	296.00	121.13	[116.06, 127.00]	+1.000	<0.0001
`random`	172.59	31.84	[23.14, 43.35]	+0.945	<0.0001
`round_robin`	163.39	28.62	[22.64, 39.73]	+0.994	<0.0001
`heuristic`	121.39	13.25	[12.15, 14.95]	–	–
`optimal`	123.29	19.15	[12.49, 30.49]	-0.022	1.0000
`PPO`	130.38	13.22	[12.34, 14.33]	+0.126	1.0000

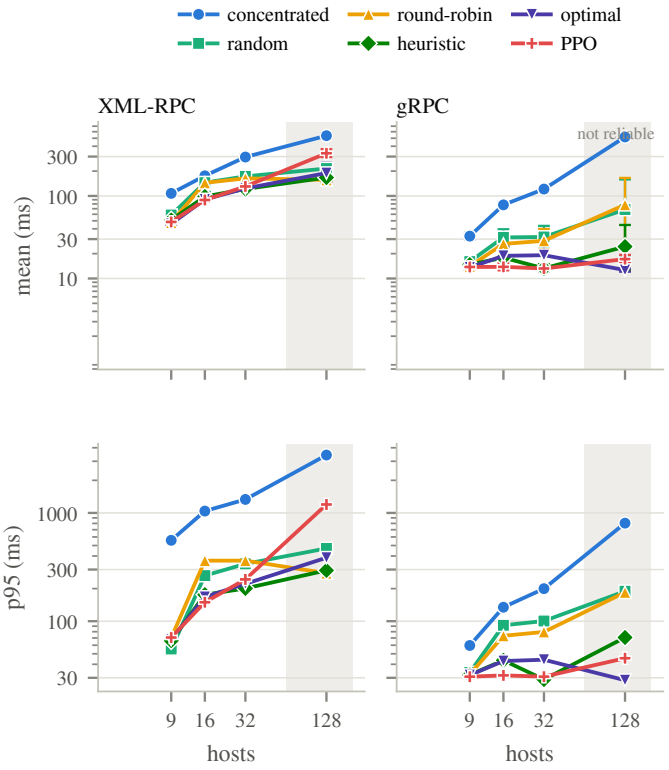


Fig. 2. Per-call service latency vs. topology scale, by transport (log axes). Top row: mean with cluster-bootstrap 95% confidence intervals. Bottom row: p95. The 9-host point is the homogeneous scenario (its `baseline` method, which exists only there, is omitted). The p95 row shows the degraded-host penalty `round_robin` and `random` pay at 16 and 32 hosts, which the gRPC arm separates more cleanly because its baseline latencies are an order of magnitude lower. The 128-host column is shaded: independent executions of that configuration differ several-fold on either transport, so it is not a usable latency measurement on a single machine (§V-E).

E. 128 Hosts: Structural Results Only

At 128 hosts (80 primaries) a single machine can no longer produce a trustworthy latency measurement, and we report no latency claims at this scale. The arithmetic is unforgiving: 80 primaries at burst 10 is 800 client threads, and on the gRPC arm roughly 75 backup processes with eight workers each add some 600 server threads, on eight cores. The symptom is that two independent executions of the identical gRPC configuration differ by 4.8× on `optimal` and 6.1× on `PPO`, with cluster-bootstrap intervals spanning an order of magnitude. The one effect that survives is gross concentration: `concentrated` is worse than every other method with

TABLE VII
SERVICE LATENCY VS. BACKUP LOAD ON THE XML-RPC ARM, ONE OBSERVATION PER (RUN × BACKUP).

Scenario	Load	n units	Mean (ms)	95% CI
Homog.	1	354	45.23	[44.60, 45.89]
	2	33	68.07	[65.80, 70.35]
	3	15	96.94	[90.51, 104.25]
	4	15	126.20	[117.85, 135.44]
16-host	1	678	104.56	[98.86, 110.46]
	2	21	107.14	[100.41, 113.65]
	3	15	229.03	[115.11, 349.47]
	9	15	190.79	[181.37, 199.20]

TABLE VIII
128 HOSTS (80 PRIMARIES, 16 DEGRADED): STRUCTURAL OUTCOME OF THE PRODUCED MAPPING. PER-CALL LATENCY IS NOT A USABLE MEASUREMENT AT THIS SCALE ON A SINGLE MACHINE (§V-A), BUT THE MAPPING ITSELF IS EXACT AND HARDWARE-INDEPENDENT.

Method	Distinct backups used (of 80)	
	XML-RPC	gRPC
`concentrated`	2–2	2–2
`random`	54–62	54–62
`round_robin`	80–80	80–80
`heuristic`	74–76	74–76
`optimal`	80–80	80–80
`PPO`	24–30	24–30

$r = 1.000$ on both transports and in both executions, and on the XML-RPC arm it is the only configuration where client retries contribute measurably at all (Table IX: 3.53% of calls retry, and 19.6 of the 35.1 s wall-clock p99 is the client's own backoff sleep rather than backup delay).

What remains valid at this scale is the *structure* of the produced mapping, which is computed from the assignment and does not depend on timing at all (Table VIII). Here the finding is unchanged from the smaller scales and is the one this scenario was built to test: the deterministic heuristic spreads 80 primaries across nearly as many distinct backups, while the learned policies do not. Validating the latency consequence of that structure at this scale requires genuinely distributed hardware, which we identify as the most important remaining work (§VI).

Why the learned policies concentrate. The original `rl_model_scaled128` (400,000 timesteps, `stable-baselines3`'s default entropy coefficient of 0 and a $[64, 64]$ network) converged to a near-degenerate policy, using only 1–5 distinct backups out of 80. With no entropy bonus nothing discourages a policy from locking onto one action once it looks decent on average, compounded by a $[64, 64]$ network's limited capacity to represent a policy conditioned on which of 80 one-hot-encoded primaries is active. Retraining with entropy 0.02, a $[256, 256]$ network and 1M timesteps widened the spread substantially, and both checkpoints ship.

TABLE IX
128 HOSTS, XML-RPC: BACKUP SERVICE TIME VS. CLIENT RETRY WAIT. ONLY AT THIS SCALE DOES CLIENT-SIDE BACKOFF CONTRIBUTE MEASURABLY; AT 9/16/32 HOSTS NO CALL RETRIES.

Method	Service p99	Wall p99	Retry wait	Calls retried (%)
`concentrated`	15485.05	35081.66	19596.61	3.53
`random`	1434.86	1434.90	0.04	0.00
`round_robin`	1189.76	1189.76	0.00	0.00
`heuristic`	1203.12	1203.12	0.00	0.00
`optimal`	1263.54	1263.54	0.00	0.00
`PPO`	7210.80	7281.93	71.13	0.27

The reward's anti-concentration term, and what it is worth. The reward already penalizes concentration directly: as §III states, a choice that makes host c the ℓ-th primary's backup is penalized by $-2(\ell - 1)$. The question is therefore not whether to add such a term but whether the one present is sufficient, which Table X answers by sweeping its strength. Two training seeds per setting are used, since RL outcomes vary enough between seeds that a single run per setting is not evidence.

The term does deliver its intended benefit, and how much it matters depends sharply on scale. At 32 hosts the effect is real but modest and saturates almost at once: removing it costs about three distinct backups, and restoring even a small penalty recovers most of what a large one achieves, so the published strength of 2 sits on a flat part of the curve. At 128 hosts it is not optional. With the penalty removed the policy collapses onto 3.2 distinct backups of 80 at a peak load of 55, which is the degenerate behaviour §V-E describes; with the published strength it reaches 13.7. The term is thus doing substantial work at the scale where the policy is under the most pressure, and very little where it is not.

What the term does *not* do, at either scale, is close the gap to the deterministic heuristic, which reaches a wider spread at no training cost: 20 of 20 distinct backups at 32 hosts against PPO's best of 18.0, and 74–76 of 80 at 128 hosts against PPO's 13.7. Host-quality discrimination and load-balancing precision are acquired at different rates as the action space grows, and a scalar penalty on the immediate reward does not change that: the agent is credited for concentration only at the step where it occurs, while the consequences accrue over the rest of the episode. A formulation that scores the assignment as a whole, rather than one primary at a time, is the more promising direction, and we leave it to future work (§VII).

The 128-host response is also *non-monotone*, and this replicates across both training seeds: a penalty of 10 is far worse than the published 2 on either seed (5.9 distinct backups against 13.7) and barely better than no penalty at all. The magnitude varies between seeds (3.2 and 8.6) while the direction does not, so we state the ordering and not the size. A plausible mechanism is that with 80 primaries sharing the pool some repetition is unavoidable, so at a large penalty the agent accumulates substantial negative reward it cannot avoid, which

TABLE X

ABLATION OF THE REWARD'S ANTI-CONCENTRATION PENALTY. PPO, EVALUATED ON FIVE RESOURCE DRAWS PER SETTING. 32 HOSTS: 150K TIMESTEPS, AVERAGED OVER TWO TRAINING SEEDS. 128 HOSTS: 400K TIMESTEPS, $[256, 256]$ NETWORK, ENTROPY 0.02, ALSO AVERAGED OVER TWO TRAINING SEEDS. NO SETTING ROUTED A PRIMARY TO A DEGRADED HOST.

	32 hosts (of 20)		128 hosts (of 80)
Penalty	Distinct	Peak load	Distinct
Peak load			
0 (no shaping)	15.1	2.6	3.2
55.4			
0.5	17.3	2.1	–
–			
2 (published)	17.3	2.0	**13.7**
40.2			
5	**18.0**	2.1	–
–			
10	16.8	2.1	5.9
62.6			
`heuristic`	**20.0**	**1–2**	**74–76**
2			

TABLE XI

PPO VS. DQN POLICY STRUCTURE, 5 CONTEXT SEEDS PER SCALE. "DISTINCT" IS THE RANGE OF DISTINCT BACKUPS A POLICY ASSIGNS ITS PRIMARIES ACROSS SEEDS (HIGHER IS A WIDER SPREAD), "PEAK" THE LARGEST NUMBER OF PRIMARIES PLACED ON ANY ONE BACKUP (LOWER IS BETTER), AND "DEGRADED" THE ASSIGNMENTS LANDING ON A DELIBERATELY DEGRADED HOST, OUT OF 5 SEEDS $\times\ |P|$.

Scale	Algorithm	Distinct	Peak	Degraded
9-host	PPO	5–5 of 5	1–1	0/25
	DQN	5–5 of 5	1–1	0/25
16-host	PPO	10–10 of 10	1–1	0/50
	DQN	9–10 of 10	1–2	0/50
32-host	PPO	15–19 of 20	2–2	0/100
	DQN	12–16 of 20	2–3	1/100
128-host	PPO	24–30 of 80	9–28	0/400
	DQN	**1–3 of 80**	**45–80**	**172/400**

destabilizes value estimation. The practical reading is that this lever has a genuine optimum rather than a monotone response, and that tuning it is therefore another per-scale burden the deterministic heuristic does not carry.

F. Cross-Algorithm Check: DQN

The load-balancing gap of §V-E is measured on one algorithm, which leaves open whether it reflects PPO or the problem formulation. To separate these we repeat the evaluation with Deep Q-Networks (DQN) [27], a deliberately different design point: off-policy with a replay buffer rather than on-policy, value-based rather than actor-critic, and exploring by epsilon-greedy decay rather than an entropy bonus. DQN is trained and evaluated through the same environment, reward, and inference path as PPO, so only the learning algorithm differs.

We score both policies *structurally*, by degraded-host assignments, distinct backups used, and maximum per-backup load (Table XI). These are properties of the produced mapping rather than of RPC timing, which matters here for two reasons. First, load-balancing precision is itself a structural property, so this measures the quantity in question directly rather than through its latency consequence, and Table VII has already established backup load as what drives latency under contention. Second, structural metrics are hardware-independent, so unlike per-call latency they reproduce exactly on any machine.

Up to 32 hosts the two algorithms behave alike. Both spread across nearly every available backup at 9 and 16 hosts, and at 32 hosts DQN stays within a few backups of PPO at a comparable peak load, with a single incidental degraded-host assignment. Reaching that required the same class of intervention PPO needed at 128 hosts: with library-default settings DQN collapsed to 2–4 distinct backups of 20, and widening exploration (exploration fraction $0.5 \to 0.7$, final epsilon $0.05 \to 0.15$), enlarging the replay buffer ($50\text{k} \to 100\text{k}$), and training longer ($100\text{k} \to 150\text{k}$ steps) recovered the spread reported here.

At 128 hosts DQN concentrates far more sharply than PPO. It funnels the 80 primaries onto 1–3 distinct backups with up to 80 primaries on a single host, and routes 172 of 400 assignments onto degraded hosts. PPO, with its entropy- and capacity-scaled configuration, avoids degraded hosts entirely in the same comparison while still spreading across only 24–30 backups, against the heuristic's 74–76 at a maximum load of 2 (§V-E).

What this establishes. Two algorithms that differ in exploration mechanism, policy representation, and update rule both concentrate load as the action space grows, and both respond to the same widened-exploration remedy. The behavior is therefore better attributed to the problem formulation, in particular a reward that labels host quality explicitly but discourages concentration only implicitly, than to any single algorithm. It also sharpens the practical reading of §V-E: the deterministic heuristic reaches a wider spread than either learned policy with no training, no per-scale tuning, and no exposure to this failure mode.

G. Degradation Realism, Weight Sensitivity, and What the Scoring Terms Buy

§III fixes the heuristic's scoring weights at $(w_h, w_k, w_\ell, w_d) = (0.35, 0.35, 0.20, 0.10)$ and its load-decay mechanics ($\text{cap} -= 20$; $\text{health} *= 0.95$) by intuition. To test whether those are principled, we sweep 287 weight configurations (286 points on a 0.1-step grid over the 4-simplex, plus the published default, which does not fall on that grid) across four topologies with five context-seed

draws each. The sweep is *structural*: it evaluates the produced mapping directly, with no RPC calls, so it is exact and reproducible.

How degraded hosts are constructed determines what the sweep can tell us. Our first model marked a host degraded by pinning it to one extreme point: health 0.05 against a healthy draw of $U[0.5, 1.0]$, and a capacity ratio of 0.033 against a healthy ratio of 1.0. Degraded hosts were thus separated from healthy ones by an order of magnitude on *both* scoring features at once, and every degraded host was identical. Under that model all 287 configurations at all four topologies achieved 0% degraded-host usage, including a configuration placing 100% of its weight on hop-count and therefore unable to see host quality at all. A result that holds even for a scoring function with no discriminative power is a property of the benchmark, not of the method, so we do not claim it.

We therefore introduce a second, harder model. Severity is drawn continuously rather than pinned, so some hosts are genuinely marginal; degradation has five kinds (CPU starvation, failing health, capacity exhaustion, storage pressure, and link congestion), each driving a different subset of the features, so a selector reading only one signal no longer catches everything; degradation is partial, so a host can be slow but healthy or healthy but nearly full; and healthy hosts carry a baseline utilisation, so remaining capacity is a continuous measurement rather than a flag that only degraded hosts trip. The last point also addresses a formulation gap: previously every non-degraded host presented as completely idle regardless of how many primaries it already backed.

Under realistic degradation the sweep discriminates sharply, and one failure mode defeats the published scoring function. At 32 and 128 hosts, 80 and 111 of 286 configurations route at least one primary to a degraded host. The heuristic at its default weights went from 0.0 to 0.8 degraded assignments per run at 32 hosts. The cause is specific: the `link_congested` kind adds 20 ms to every call while reading *above average* on health, capacity, CPU and storage, because its degradation lives in the network path rather than in any resource gauge. The two features that could reveal it, latency and hop-count, are derived from the topology graph, and in a uniform-hop star every backup-eligible host is exactly two hops from every primary, so both normalize to zero and carry no signal whatsoever.

Scoring against measured RTT restores the guarantee. A production orchestrator does not have to infer path latency from hop-count; it has active RTT probing, which is precisely the telemetry §III maps the latency feature onto. Replacing the topology-derived latency term with measured round-trip time, which includes whatever delay the candidate is currently adding, returns degraded-host assignments to 0.0 at 32 hosts and 0.1 at 128. The single residual case is a host at severity 0.12 adding 11.5 ms, which is genuinely marginal rather than a miss.

This inverts which scoring terms matter. Table XII reports, for each term, how often a configuration that zeroes it routes a primary to a degraded host. The latency term, evaluated against measured RTT, is what provides degraded-host avoidance: zeroing it causes 55 of 66 configurations to fail at both scales. Zeroing the health term causes *none* to fail. Under the binary model this question had no answer at all, since every configuration passed. The published default survives with zero degraded assignments at every scale, but now for an identifiable reason, namely its non-zero latency weight, rather than because every configuration passes.

TABLE XII
CONFIGURATIONS (OF 66 THAT ZERO THE GIVEN TERM) ROUTING AT LEAST ONE PRIMARY TO A DEGRADED HOST, REALISTIC DEGRADATION, 5 SEEDS. UNDER THE BINARY DEGRADATION MODEL EVERY ENTRY IS 0, AND THE SWEEP CANNOT DISTINGUISH BETWEEN WEIGHT CHOICES.

Term zeroed	32 hosts	128 hosts
w_h (health)	0/66	0/66
w_k (capacity)	12/66	35/66
w_ℓ (latency, measured RTT)	**55/66**	**55/66**
w_d (hop-count)	18/66	25/66
Any configuration: 80/286 and 111/286 fail.		
Published default $(0.35, 0.35, 0.20, 0.10)$: 0 and 0.		

One caveat bounds this. Every degradation kind in our model carries some latency signature, which is realistic (a degraded host does serve more slowly) but makes measured RTT close to a sufficient statistic in this benchmark. A failure mode with no latency signature, such as a host that is currently fast but about to fail, would defeat it and would need the health signal instead. The defensible claim is therefore not that latency is the only term worth having, but that a scoring function restricted to static resource gauges is blind to network-path degradation, and that the remedy is measured rather than inferred path latency.

Load-balancing quality is governed by the capacity decay term. Separately from host quality, maximum per-backup load ranges across the sweep from perfect (one primary per backup) to severe (79 of 80 primaries sharing one backup at 128 hosts, when capacity and health weights are both zero). The capacity term, realised as the flat -20 decrement applied after each assignment, is the dominant lever: even a small non-zero capacity weight reaches near-optimal balance at every scale. The health decay ($\times 0.95$) is a weaker lever toward the same end. Sweeping both decay constants (the decrement over $\{5, 10, 20, 40, 80\}$ and the health factor over $\{0.80, 0.90, 0.95, 0.99, 1.00\}$, 100 cells) shows load-balancing to be a smooth, monotone function of both: holding the health decay at 0.95, raising the decrement through $5, 10, 20, 40, 80$ lowers mean maximum per-backup load through $2.60, 2.00, 2.00, 1.00, 1.00$. The published decrement of 20 is thus a deliberately conservative operating point within one unit of the perfect-spread optimum, leaving a small bounded amount of load-balancing unclaimed rather than spreading maximally aggressively, which on a pool with

finer quality variation could trade host quality for balance.

H. Decentralized Hash Baselines, a Fairer Round-Robin, and Topology Diversity

We broaden the baseline set along three axes the load-balancing and distributed-storage literature would expect, and re-examine the findings structurally (degraded-host hits and maximum per-backup load, 5-seed average, no RPC calls) in the heterogeneous scenarios where host quality differs (Table XIII).

Decentralized hash placement (HRW and consistent hashing). We add three decentralized baselines in the style well established in distributed storage (CRUSH [16], Dynamo [17]): plain rendezvous hashing (`hrw`), health/capacity-weighted rendezvous hashing (`weighted_hrw`), and Dynamo-style weighted consistent hashing (`weighted_chash`). Plain HRW is health/capacity-unaware, and like round-robin its degraded-host exposure is uncontrolled: it routes primaries to degraded hosts at 16 and 128 hosts and misses them at 32 only by hash coincidence. Both *weighted* variants avoid degraded hosts entirely (0 across all scales), confirming that *quality-awareness, not centralization, is what buys degraded-host avoidance*. However, both are coordination-free (each primary hashes independently, with no load penalty propagated between assignments), so they spread load only statistically: mean maximum per-backup load is 3.0/3.6/2.2 (weighted HRW) and 2.8/3.0/3.2 (weighted consistent hashing) at 16/32/128 hosts, against the greedy heuristic's 1.4/2.0/2.0 and the Hungarian optimum's 1.0. A single coordinated greedy pass therefore load-balances materially tighter than *either* decentralized weighted-hash family at equal degraded-host avoidance, and the exact assignment is tighter still.

A fairer round-robin. The deterministic round-robin's degraded-host exposure depends on Python's alphabetical host-name sort, which happens to miss the degraded range at 128 hosts (§V-C). Adding a randomized-offset round-robin (`rr_random`) removes that artifact: with a random starting offset it hits degraded hosts at *every* scale (1.2/2.6/10.8 primaries at 16/32/128 hosts, tracking the degraded fraction), confirming that the deterministic variant's apparent 128-host safety was pure naming coincidence, and that any quality-blind load-spreader is exposed to degraded hosts in proportion to how many there are. This strengthens rather than weakens the core claim: the feature-aware methods avoid degraded hosts *by construction* at every scale, a guarantee no round-robin variant offers.

Topology diversity. To test whether the findings depend on the single-hub star, we repeat the comparison on multi-hub topologies (three hubs in a line, hosts distributed across them, so host-to-host hop-count and latency now vary). The structural outcome is unchanged: at 16-host/3-hub and 32-host/3-hub the heuristic still avoids degraded hosts entirely at a maximum load of 1.4 and 2.0 (identical to the single-hub stars), the weighted-hash baselines still avoid degraded hosts but spread loosely (max load ≈ 3), and the fair round-robin still hits degraded hosts (1.2 and 2.6). Because degraded-host avoidance and load-balancing are driven by host *quality* rather than graph *shape*, the core findings are robust to topology structure; a fuller sweep over irregular, multi-hub, and uneven-fan-out graphs remains future work (§VI).

TABLE XIII
DECENTRALIZED HASH BASELINES AND A FAIRER ROUND-ROBIN VS. THE FEATURE-AWARE METHODS, 5-SEED AVERAGE, HETEROGENEOUS STAR SCENARIOS (MULTI-HUB RESULTS ARE IDENTICAL; SEE TEXT). "DEG." = MEAN PRIMARIES ROUTED TO A DEGRADED HOST (LOWER IS BETTER); "LOAD" = MEAN MAX PER-BACKUP LOAD (LOWER IS A TIGHTER SPREAD).

	16-host		32-host		128-host	
Method	Deg.	Load	Deg.	Load	Deg.	Load
round_robin	2.0	1.0	4.0	1.0	0.0	1.0
rr_random	1.2	1.0	2.6	1.0	10.8	1.0
hrw	2.0	3.0	0.0	3.0	9.0	2.0
weighted_hrw	0.0	3.0	0.0	3.6	0.0	2.2
weighted_chash	0.0	2.8	0.0	3.0	0.0	3.2
heuristic	**0.0**	**1.4**	**0.0**	**2.0**	**0.0**	**2.0**
optimal	0.0	1.0	0.0	1.0	0.0	1.0

I. Selection Overhead

"Elastic scaling" requires the selection algorithm itself to be fast: a method that matches the optimal bound on placement quality but takes seconds to compute is a materially worse deployment story than one that is comparably fast. Table XIV reports the mean wall-clock time to compute one full mapping, across all four topology sizes (30 repeats each, first call excluded as warmup), and Figure 3 plots the same data on log–log axes. Even the slowest method computes a full 128-host mapping in about 60 ms (`PPO`, 61.6 ms mean), and every non-learned method in roughly 10 ms or less, negligible relative to any realistic failure-detection interval (typically hundreds of milliseconds to seconds in NF failure-recovery literature [28], [29]). `PPO` is consistently slowest (one PPO forward pass per primary, plus framework call overhead); its jump at 128 hosts is the direct cost of the wider $[256, 256]$ network needed to fix the policy collapse described in §V-E, on top of the growth in primary count. `heuristic` and `optimal` grow roughly linearly/superlinearly with host count (the Hungarian algorithm is $O(n^3)$) but remain near 10 ms at 128 hosts. None of the feature-aware methods introduce a selection-latency cost that would offset their placement-quality advantage in practice.

J. Throughput and Failure Rate

Tables XV and XVI complement service latency with the other two metrics DEFT's original evaluation reports: throughput and packet drop. We measure each primary's burst workers for a fixed 3-second wall-clock window (rather than a fixed call count) and report successful calls/second and the percentage of calls that fail even after retry, with one observation per run and the same paired analysis as §IV-D. This testbed is RPC-level (TCP), not DEFT's own packet-level input-buffer-overflow mechanism, which requires the Mininet/stamped-

TABLE XIV
SELECTION OVERHEAD (MS) PER FULL MAPPING COMPUTATION, MEAN OF 30 REPEATS (FIRST CALL EXCLUDED AS WARMUP).

Method	9 hosts	16 hosts	32 hosts	128 hosts
concentrated	0.006	0.021	0.035	0.600
random	0.028	0.043	0.054	0.567
round_robin	0.005	0.007	0.009	0.055
heuristic	0.050	0.145	0.600	10.277
optimal	0.081	0.271	0.860	9.968
PPO	6.345	5.065	10.605	61.636
DQN	1.404	6.209	6.217	40.080

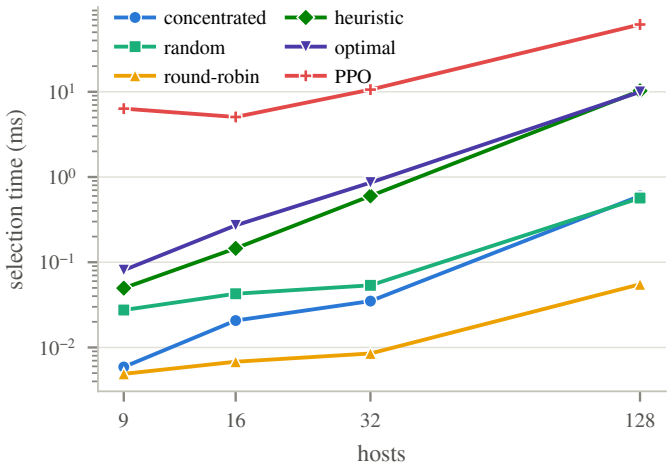


Fig. 3. Selection overhead per full mapping computation (log–log axes; data of Table XIV). Even the slowest method (PPO, 49.6 ms at 128 hosts) remains well below realistic failure-detection intervals.

packet path this testbed deliberately bypasses; "failure rate" here is evidence of the same underlying phenomenon (an overloaded backup failing to keep up with demand), not a literal reproduction of DEFT's packet-drop metric.

At homogeneous scale we draw no conclusion from throughput. Each method contributes only 15 aggregate measurements, against the tens of thousands of individual calls behind each latency test, and *none* of the 21 pairwise comparisons survives Holm-Bonferroni correction; the confidence intervals overlap almost completely. Aggregate throughput here is dominated by background system load rather than by the mapping under test. Failure rate is the one consistent signal: concentrated's 1.78% is the highest of any method, while the feature-aware methods stay at 0.23–0.48%.

At 16 hosts, contention is severe enough that one part of the throughput picture becomes robust. concentrated is significantly out-thoughput by *every* other method ($r \in [-1.000, -0.900]$, all $p_{\text{Holm}} \leq 0.019$), and its failure rate (5.82%) is an order of magnitude above the feature-aware methods' (0.26–0.48%). Gross load concentration is therefore visible from the throughput side, not only from service latency.

The degraded-host effect is a subtler matter, and the honest characterization is that aggregate throughput resolves it only partially. PPO does significantly out-throughput round_robin ($r = -0.933$, $p_{\text{Holm}} = 0.0047$), consistent with round_robin pairing 2 of its 10 primaries with a degraded backup in every seed; yet neither heuristic nor optimal reaches significance against round_robin ($p_{\text{Holm}} \geq 0.86$) despite avoiding degraded hosts exactly as PPO does. The mechanism explains the fragility: only 2 of round_robin's 10 primaries are actually paired with a degraded backup, and their reduced individual throughput is averaged into the other 8 primaries' normal throughput at the aggregate level, leaving a signal small enough that run-to-run noise decides the verdict. Service latency is dominated by exactly the slow calls that throughput's mean-based aggregation smooths over, and resolves the same effect consistently at every scale we tested. This is why we treat latency as the primary metric throughout and throughput as corroborating rather than replacing evidence: throughput reliably resolves gross contention, but only marginally resolves the finer degraded-host effect that this paper is actually about.

TABLE XV
THROUGHPUT/FAILURE RATE, HOMOGENEOUS SCALE (15 RUNS).

Method	Throughput (calls/s)	95% CI	Failure rate (%)
baseline	523.20	[440.78, 617.89]	1.021
concentrated	530.02	[437.56, 634.27]	1.781
random	520.53	[436.37, 613.89]	0.797
round_robin	547.09	[442.24, 669.62]	0.282
heuristic	570.67	[473.06, 683.80]	0.480
optimal	549.62	[453.67, 661.73]	0.227
PPO	541.69	[461.73, 630.05]	0.315

TABLE XVI
THROUGHPUT/FAILURE RATE, 16-HOST SCALED SCENARIO (15 RUNS).

Method	Throughput (calls/s)	95% CI	Failure rate (%)
concentrated	543.84	[477.84, 616.58]	5.821
random	688.84	[626.53, 757.76]	1.224
round_robin	733.49	[677.93, 795.25]	0.428
heuristic	752.64	[689.98, 821.42]	0.480
optimal	740.11	[691.78, 790.83]	0.264
PPO	780.96	[721.91, 846.84]	0.321

VI. THREATS TO VALIDITY

RPC-isolation testbed, not the physical Mininet network (the primary external-validity gap). As described in §IV, we exercise DEFT's real backup RPC server directly rather than over a physical/emulated network path, after the available Mininet environment could not provide a genuine bandwidth constraint for contention to manifest against. This gives a direct, reproducible measurement of the contention mechanism at the cost of not validating full end-to-end network behavior (switching, real link bandwidth, packet loss) and, in particular, DEFT's own packet-level, controller-integrated data path. A packet-level evaluation over a working OVS/SDN-controller Mininet fabric, against which the RPC-level failure rate could be cross-checked with DEFT's native packet-drop metric, is the most important remaining validation, and we treat the present study as isolating the backup-selection mechanism rather than certifying end-to-end network performance.

Synthetic resource values and injected heterogeneity. Health, capacity, CPU, and storage availability are drawn from the configured ranges of §III, and heterogeneity is injected via a fixed $+50$ ms per-call delay and pinned degraded resource numbers, not measured from real workload telemetry; absolute magnitudes are therefore illustrative rather than calibrated. §III maps each synthetic feature to the real NFV telemetry it stands in for, but validating the selection against that live telemetry remains future work, and limits the ecological validity of the absolute (though not the relative) results.

RL configuration scales with the action space. A PPO agent's observation and action space is fixed to a given host count, so a differently-sized topology is served by its own trained model, and, as §V-E shows, the exploration and network-capacity budget must grow with the action space (the defaults that suffice at 16 and 32 hosts require scaling up at 128). We report the resulting behavior, ablate the reward's anti-concentration term (§V-E), and leave a broader study of RL for this problem to future work. For reproducibility, the reported RL numbers describe the specific released checkpoints; retraining under a different `stable-baselines3`/`PyTorch` version or host count yields a different but qualitatively similar policy. The same applies to DQN, which additionally required its own exploration widening at 32 hosts (§V-F).

Deterministic round-robin's degraded-host exposure is coincidental. Its exposure depends on how Python's alphabetical host-name sort aligns with the degraded hosts (hitting them at 16/32 hosts, missing at 128); the randomized-offset variant of §V-H confirms the exposure is real and proportional to the degraded fraction. This does not affect the load-bearing claim that heuristic and optimal avoid degraded hosts by construction at every scale, and that PPO does so empirically at every scale tested, which no round-robin variant guarantees.

Two transports, not every transport. Evaluating over both XML-RPC and gRPC establishes that the placement findings are not an artifact of one RPC stack, and gRPC is representative of how production NFV systems carry state replication. It is not, however, the whole space: kernel-bypass paths (DPDK ring buffers, RDMA) remove the kernel networking stack entirely and would exhibit queueing of a different shape again. The relative ordering of selection methods is driven by how many primaries share a backup, which we expect to be robust, but absolute magnitudes are specific to the transport measured.

Single physical machine, and its hard limit at 128 hosts. All "hosts" are OS processes on one machine. Below 32 hosts this is a controlled way to isolate server-side contention, and the two transports agree. At 128 hosts it fails outright: independent executions of the identical configuration differ several-fold, so we report only structural results there (§V-E). Validating the latency consequences of placement at that scale needs genuinely distributed hardware with real NICs and switches, and is the most important remaining work.

Absolute latencies are not reproducible across executions. Ephemeral-port TIME_WAIT pressure shifts the mean of every method by of order 17 ms between otherwise identical executions (§V-A). The paired design cancels this, since the drift is common-mode, but it sets a resolution floor: we make no claim about differences of a few milliseconds, and two such claims that appeared significant in a single execution did not survive replication (§V-A, §V-C).

Scale sweep is four points, not a continuous curve. All four are now run at an identical protocol and harness version, but a continuous sweep would be needed for a rigorous asymptotic scalability claim.

Topology diversity is partially, not exhaustively, covered. Beyond the single-hub stars, we evaluate multi-hub topologies (§V-H), where host-to-host hop-count and latency vary, and the structural findings are unchanged, because degraded-host avoidance and load-balancing are driven by host quality rather than graph shape. Our context-seed sweep still varies only synthetic *resource* draws per seed, however; the graph shapes tested (single- and multi-hub) remain regular, and genuinely irregular graphs (uneven fan-out, heterogeneous hub connectivity, real datacenter topologies) are untested. A fuller sweep over such graphs, together with the continuous scale sweep noted above, would strengthen the generalization claim.

Two analyses are evaluated structurally rather than by measured latency. The weight and decay sweeps (§V-G) and the PPO-vs-DQN comparison (§V-F) both report properties of the produced mapping (degraded-host assignments, maximum per-backup load, distinct backups used) instead of per-call RPC latency. This keeps them exact, hardware-independent, and cheap enough to sweep 287 configurations, and it measures load-balancing precision directly rather than through its latency consequence; our mechanism check (Table VII) establishes that backup load is what drives latency under contention. The cost is that neither the ranking of weight configurations nor DQN's latency under contention has been verified end to end against measured latency alongside the other methods.

VII. Conclusion and Future Work

Every major NF state-replication system we surveyed (OpenNF, Split/Merge, StatelessNF, S6, Pico Replication, FTMB, REINFORCE, and DEFT) does not treat backup-node assignment as a decision with measurable consequences. We have closed this gap concretely with iDEFT, designing and rigorously evaluating three backup-selection strategies (an exact Hungarian-optimal bound, a multi-criteria heuristic, and a PPO policy) against DEFT's own static mapping and three naive baselines, by driving DEFT's real, unmodified 2PC implementation under controlled contention across four topology scales with statistically rigorous, multi-seed testing. Feature-aware methods reliably avoid degraded backup hosts and substantially reduce mean and p95 service latency once hosts differ in quality: at both 16 and 32 hosts the greedy heuristic beats blind round-robin in every one of fifteen paired runs ($r = 1.00$, $p_{\mathrm{Holm}} \leq 0.0012$), while remaining statistically indistinguishable from the Hungarian optimum at every scale we tested. Reinforcement learning (PPO) matches this performance up to 32 hosts and, at 128 hosts, avoids degraded hosts completely while its load-balancing precision

is lower than the heuristic's at that scale. A cross-algorithm check with DQN reproduces that concentration behavior at scale, and more sharply, under the same widened-exploration remedy, which attributes the gap to the problem formulation rather than to one algorithm. We report this measured behavior, and ablate the reward's anti-concentration penalty: it helps, but its benefit saturates at a small strength and never reaches the untrained heuristic's spread, which points at the per-step credit assignment rather than at the penalty's magnitude. A systematic sensitivity analysis over the heuristic's scoring weights and decay mechanics further shows that its load-balancing quality is governed by the remaining-capacity term (whose -20 decrement is the load-spreading mechanism), and that its degraded-host avoidance depends on how degradation is modelled. Against an all-or-nothing degraded host every weight configuration succeeds and the sweep is uninformative; against continuous, partial, multi-kind degradation the sweep discriminates sharply, and a host degraded only in its network path defeats a scoring function built on static resource gauges. Measured round-trip time, which a production orchestrator already collects, restores the guarantee. The published default weighting survives both models, but under the harder one it survives for an identifiable reason rather than because everything does.

We also report a methodological result. Evaluating the same methods over two transports shows that DEFT's shipped XML-RPC path inflates the measured placement effect by roughly $4.3\times$, because an HTTP/1.0 server with a five-deep accept backlog converts a degraded host's modest per-call service penalty into accept-queue overflow and kernel retransmission waits; under gRPC the tail is ordinary queueing and p99 becomes meaningful again. Placement studies built on connection-per-call RPC harnesses are subject to the same inflation. Two further limits are worth recording: ephemeral-port pressure shifts absolute latencies between identical executions by more than most differences one might wish to claim, and a single machine cannot resolve latency at 128 hosts on either transport. A run-level paired design, in which the independent unit is an execution rather than an individual call, absorbs the former; only distributed hardware addresses the latter. We recommend that design, and two independent replications per configuration, for anyone evaluating placement policies on a comparable harness.

Future work includes: an RL formulation that scores the assignment as a whole rather than one primary at a time, since the ablation shows a per-step concentration penalty saturates before reaching the heuristic's spread; validating these findings over a physical or emulated network path with genuine bandwidth constraints, and over a production-grade asynchronous transport (gRPC, RDMA) whose queueing behavior differs in kind from XML-RPC's; extending the multi-seed evaluation to vary topology structure, not just resource values; and a continuous scalability sweep across an uninterrupted range of host counts.

ACKNOWLEDGMENT

This work builds directly on the DEFT system [5] and its publicly available implementation.

REFERENCES

[1] B. Han, V. Gopalakrishnan, L. Ji, and S. Lee, "Network function virtualization: Challenges and opportunities for innovations," *IEEE communications magazine*, vol. 53, no. 2, pp. 90–97, 2015.

[2] S. Rajagopalan, D. Williams, and H. Jamjoom, "Pico replication: A high availability framework for middleboxes," in *Proceedings of the 4th annual Symposium on Cloud Computing*, 2013, pp. 1–15.

[3] J. Sherry, P. X. Gao, S. Basu, A. Panda, A. Krishnamurthy, C. Maciocco, M. Manesh, J. Martins, S. Ratnasamy, L. Rizzo *et al.*, "Rollback-recovery for middleboxes," in *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, 2015, pp. 227–240.

[4] S. G. Kulkarni, G. Liu, K. Ramakrishnan, M. Arumaithurai, T. Wood, and X. Fu, "Reinforce: Achieving efficient failure resiliency for network function virtualization based services," in *Proceedings of the 14th International Conference on emerging Networking EXperiments and Technologies*, 2018, pp. 41–53.

[5] M. M. Shahriyar, G. Saha, B. Bhattacharjee, and R. Reaz, "Deft: distributed, elastic, and fault-tolerant state management of network functions," in *2023 19th International Conference on Network and Service Management (CNSM)*. IEEE, 2023, pp. 1–7.

[6] A. Gember-Jacobson, R. Viswanathan, C. Prakash, R. Grandl, J. Khalid, S. Das, and A. Akella, "Opennf: Enabling innovation in network function control," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 4, pp. 163–174, 2014.

[7] S. Rajagopalan, D. Williams, H. Jamjoom, and A. Warfield, "{Split/Merge}: System support for elastic execution in virtual middleboxes," in *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*, 2013, pp. 227–240.

[8] M. Kablan, A. Alsudais, E. Keller, and F. Le, "Stateless network functions: Breaking the tight coupling of state and processing," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 2017, pp. 97–112.

[9] S. Woo, J. Sherry, S. Han, S. Moon, S. Ratnasamy, and S. Shenker, "Elastic scaling of stateful network functions," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, 2018, pp. 299–312.

[10] M. Szalay, M. Nagy, D. Géhberger, Z. Kiss, P. Mátray, F. Németh, G. Pongrácz, G. Rétvári, and L. Toka, "Industrial-scale stateless network functions," in *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*. IEEE, 2019, pp. 383–390.

[11] I. Stoica, R. Morris, D. Liben-Nowell, D. R. Karger, M. F. Kaashoek, F. Dabek, and H. Balakrishnan, "Chord: a scalable peer-to-peer lookup protocol for internet applications," *IEEE/ACM Transactions on networking*, vol. 11, no. 1, pp. 17–32, 2003.

[12] J. S. Plank, M. Beck, G. Kingsley, and K. Li, *Libckpt: Transparent checkpointing under unix*. Computer Science Department, 1994.

[13] C. Cachin, S. Schubert, and M. Vukolić, "Non-determinism in byzantine fault-tolerant replication," *arXiv preprint arXiv:1603.07351*, 2016.

[14] G. L. Santos, P. T. Endo, T. Lynn, D. Sadok, and J. Kelner, "A reinforcement learning-based approach for availability-aware service function chain placement in large-scale networks," *Future Generation Computer Systems*, vol. 136, pp. 93–109, 2022.

[15] J. Pei, P. Hong, M. Pan, J. Liu, and J. Zhou, "Optimal vnf placement via deep reinforcement learning in sdn/nfv-enabled networks," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 2, pp. 263–278, 2019.

[16] S. A. Weil, S. A. Brandt, E. L. Miller, and C. Maltzahn, "Crush: Controlled, scalable, decentralized placement of replicated data," in *Proceedings of the 2006 ACM/IEEE conference on Supercomputing*, 2006, pp. 31–es.

[17] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles*, 2007, pp. 205–220.

[18] H.-K. Lim, I. Ullah, Y.-H. Han, and S.-Y. Kim, "Reinforcement learning-based virtual network embedding: A comprehensive survey," *ICT Express*, vol. 9, no. 5, pp. 983–994, 2023.

[19] N. Saha, M. Zangooei, M. Golkarifard, and R. Boutaba, "Deep reinforcement learning approaches to network slice scaling and placement: A survey," *IEEE Communications Magazine*, vol. 61, no. 2, pp. 82–87, 2023.

[20] G. Kibalya, J. Serrat, J.-L. Gorricho, D. G. Bujjingo, J. Sserugunda, and P. Zhang, "A reinforcement learning approach for placement of stateful virtualized network functions," in *2021 IFIP/IEEE International Symposium on Integrated Network Management (IM)*. IEEE, 2021, pp. 672–676.

[21] M. Avgeris, A. Leivadeas, and I. Lambadaris, "A reinforcement-learning self-healing approach for virtual network function placement," in *NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2023, pp. 1–5.

[22] H. W. Kuhn, "The hungarian method for the assignment problem," *Naval research logistics quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.

[23] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright *et al.*, "Scipy 1.0: fundamental algorithms for scientific computing in python," *Nature methods*, vol. 17, no. 3, pp. 261–272, 2020.

[24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[25] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[26] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024.

[27] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[28] P. Gill, N. Jain, and N. Nagappan, "Understanding network failures in data centers: measurement, analysis, and implications," in *Proceedings of the ACM SIGCOMM 2011 Conference*, 2011, pp. 350–361.

[29] R. Potharaju and N. Jain, "Demystifying the dark side of the middle: a field study of middlebox failures in datacenters," in *Proceedings of the 2013 conference on Internet measurement conference*, 2013, pp. 9–22.